FIT2109 Computer Science Workshop

Week 1: Introduction to the Shell

Yongqiang Tian

Department of Software Systems and Cybersecurity
Faculty of IT, Monash University

Acknowledgement

I wish to acknowledge the people of the Kulin Nations, on whose land we are gathered today. I pay my respects to their Elders, past and present.

Why FIT2109 starts with the shell

The shell is not here to replace the GUI

GUIs are excellent for visible, one-off, standard tasks.

The shell is the layer underneath modern software work

When setup fails, tools need configuration, logs need inspection, a server is remote, or a task must be repeated, the shell becomes the common control surface.

Why learn this in a GUI world?

A GUI is strongest when

- the task is already designed for you,
- the state is visible,
- you only need to do it once,
- nothing has gone wrong.

The shell is strongest when

- the task must be repeated,
- the system state must be inspected,
- the machine is remote,
- something has gone wrong.

The shell connects the whole unit

Shell $\rightarrow$ scripting $\rightarrow$ Git $\rightarrow$ remote machines $\rightarrow$ containers

$\rightarrow$ debugging $\rightarrow$ profiling $\rightarrow$ automation $\rightarrow$ agentic coding

Week 1 goal

Build practical understanding of commands, files, paths, permissions, and pipelines.

Learning goals

By the end of this workshop, you should be able to:

- Explain what the shell prompt, working directory, and pathnames mean.
- Describe the filesystem as a tree of directories and files.
- Use core commands to inspect and change the filesystem.
- Distinguish a stored program from a running process.
- Interpret simple permission strings and explain why a script may not run.
- Explain `$PATH`, standard streams, redirection, pipes, and exit status.

Your shell: bash, zsh, and why it matters

This course uses Bash

Scripts and examples use `#!/usr/bin/env bash`. Bash is the default on most Linux systems and WSL.

macOS defaults to Zsh since macOS Catalina (2019)

You may see a `%` prompt instead of `$`. That is normal — you are in Zsh.

- For all Week 1 commands (`ls`, `cd`, `pwd`, pipes, redirection), **Bash and Zsh behave identically**.
- Differences appear in scripting syntax and some expansions — covered in Week 2.
- A script with `#!/usr/bin/env bash` always runs in Bash, regardless of your interactive shell.
- `echo "$SHELL"` tells you which shell is running interactively.

Poll Everywhere · Word Cloud

pollev.com/fit2109

What is **one word** that comes to mind when you hear 'command line'?

Concept 1: The shell interprets commands

Key idea

The shell reads a line of text, interprets it, runs a command, and reports the result.

- A command has a **name**, optional **options**, and optional **arguments**.
- The shell always has context: user, machine, current directory, environment.
- The output is often more important than the command itself.

Typical shape

```
command -option argument
```

Commands are abbreviations

The name usually tells you what the command does

When you see an unfamiliar command, its full name is often the best definition.

- `pwd` — **p**rint **w**orking **d**irectory
- `ls` — **l**ist
- `cd` — **c**hange **d**irectory
- `mkdir` — **m**ake **dir**ectory
- `cp` — **c**o**p**y
- `mv` — **m**o**v**e
- `rm` — **r**e**m**ove
- `cat` — con**cat**enate
- `chmod` — **ch**ange **mod**e
- `ps` — **p**rocess **s**tatus
- `man` — **man**ual
- `grep` — **g**lobal **r**egular **e**xpression **p**rint
- `wc` — **w**ord **c**ount

Demo 1: read the current shell context

Live on Terminal

What can we learn before changing anything?

```
whoami
hostname
pwd
echo "$SHELL"
ls
```

Student check

Which command tells us who we are? Which command tells us where we are?

Which command prints the name of the currently logged-in user?

A `pwd`

B `whoami`

C `hostname`

D `ls`

Concept 2: Your location changes what commands mean

Key idea

The shell is always "standing" in one directory. Relative pathnames are interpreted from there.

- `pwd` shows the current working directory.
- `cd` changes the current working directory.
- `ls` lists files in a directory.
- Many command mistakes are really location mistakes.

Concept 3: Files and directories form a tree

Key idea

Directories contain files and other directories. A pathname describes a route through this tree.

- The root of the tree is written as `/`.
- Your home directory is your usual personal starting point.
- A directory can have children and a parent.
- Commands such as `ls`, `cd`, and `pwd` help you inspect your place in the tree.

Concept 4: Paths tell the shell where to look

Absolute path

Starts from the root of the filesystem.

`/Users/student/fit2109/week1`

Relative path

Starts from the current working directory.

`data/raw`

- `.` means current directory.
- `..` means parent directory.
- `~` means your home directory.

Demo 2: move through the filesystem

Live on Terminal

Watch how `pwd` changes after each `cd`.

```
mkdir -p ~/fit2109/week1/data/raw
cd ~/fit2109/week1
pwd
cd data
pwd
cd raw
pwd
cd ..
pwd
cd ~
pwd
```

Activity 1: predict the destination

Starting point

```
/Users/student/fit2109/week1/data/raw
```

In pairs

Where do these commands take you?

1. `cd ..`
2. `cd ../..`
3. `cd ~/fit2109/week1`
4. `cd ./data/raw`

Type the shell command that moves **up one directory** level.

Concept 5: Commands inspect or change filesystem state

Key idea

Files and directories are state. Commands either inspect state or change state.

- `mkdir`: create directories
- `touch`: create/update a file
- `cp`: copy
- `mv`: move or rename
- `rm`: remove
- `ls`: inspect

Professional habit

Inspect before and after a command when you are learning or when the change is risky.

Demo 3: create, copy, move, remove

Live on Terminal

After each command, ask: what changed?

```
cd ~/fit2109/week1
touch notes.txt
mkdir scripts backups
cp notes.txt backups/notes-copy.txt
mv notes.txt summary.txt
cp summary.txt scripts/summary-backup.txt
rm backups/notes-copy.txt
ls -R
```

Activity 2: draw the final tree

Predict

After this sequence, what files and directories exist?

```
mkdir -p demo/data
cd demo
touch notes.txt
cp notes.txt data/notes-copy.txt
mv notes.txt summary.txt
mkdir scripts
cp summary.txt scripts/summary-backup.txt
rm data/notes-copy.txt
ls -R
```

Poll Everywhere · Open-Ended

pollev.com/fit2109

In one sentence: what is **one difference** between `cp` and `mv`?

Concept 6: Permissions decide who can do what

Key idea

A file may exist, but the system still checks whether you are allowed to read, write, or execute it.

How to read the permission string

`- rwx r-x r-x` = filetype | owner | group | others

`r`: read `w`: write `x`: execute

- A `-` in place of a letter means that permission is absent.
- The first character shows file type: `-` for regular file, `d` for directory.

Demo 4: make a script executable

Live on Terminal

Why does the script fail first, then work?

```
cd ~/fit2109/week1
mkdir -p scripts
echo 'echo Hello FIT2109' > scripts/run.sh
ls -l scripts/run.sh
./scripts/run.sh
chmod u+x scripts/run.sh
ls -l scripts/run.sh
./scripts/run.sh
```

After `chmod u+x script.sh`, the **owner** can now:

A Read the file

B Write to the file

C Execute the file

D Share the file with others

Activity 3: read permissions

Two files

```
-rw-r-----  1 dennis staff 132 Mar  3 10:18 notes.txt
-rwxr-xr-x  1 dennis staff  58 Mar  3 10:20 run.sh
```

Discuss

Who can read, write, and execute each file? Why is one file runnable and the other not?

In `-rwxr-xr-`, can *others* run this file?

A Yes
B No
C Only with `sudo`
D Depends on the OS

Concept 7: Programs become processes when they run

Key idea

A program is stored code. A process is a running instance of a program.

- Running a command often starts a process.
- A process has an identifier called a PID.
- Some commands finish quickly; others keep running.
- Error messages often come from the program/process, not from the shell itself.

Demo 5: inspect running processes

Live on Terminal

The shell itself is a running process, and it can start other processes.

```
echo $$
ps -p $$ -o pid,ppid,comm
ps -ax | head
```

Student check

What is the difference between the command text you typed and the process that runs?

Concept 8: $PATH is where the shell searches

Key idea

When you type a command name, the shell does not search the whole computer. It searches directories listed in `$PATH`.

- Some commands are built into the shell, such as `cd`.
- Some commands are external programs, such as `ls`.
- If you provide a path, such as `./scripts/run.sh`, the shell uses that path directly.

Demo 6: command lookup

Live on Terminal

Compare builtins, external commands, and explicit paths.

```
echo "$PATH"
which ls
type ls
type cd
run.sh
./scripts/run.sh
```

Student check

Why might `run.sh` fail even when the file exists?

Why does `run.sh` fail even though the file exists, while `./scripts/run.sh` works?

Concept 9: Streams separate input, output, and errors

Key idea

Programs usually read input and produce output. The shell can redirect where those streams go.

- **Standard input**: where a program reads from by default.
- **Standard output**: where normal results go by default.
- **Standard error**: where error messages go by default.
- >, », <, and 2> redirect streams.

Demo 7: redirect output and errors

Live on Terminal

Watch which text appears on screen and which text goes into a file.

```
echo "first line" > notes.txt
echo "second line" >> notes.txt
cat notes.txt
sort < notes.txt
ls missing.txt > output.txt 2> errors.txt
cat output.txt
cat errors.txt
```

Concept 10: Pipes make small tools work together

- A pipe sends one program's output into another program's input.
- Each command can stay small and focused.
- A pipeline becomes a custom workflow.
- Pipeline order matters because each step transforms the stream.

Key idea

```
program A | program B | program C
```

Demo 8: answer a question from a log

Question

How many error lines are in this log, and which errors happen most often?

```
cat > app.log <<'LOG'
INFO Login ok
WARN Disk space low
ERROR Database unavailable
WARN Disk space low
ERROR Timeout contacting API
ERROR Database unavailable
LOG

grep "ERROR" app.log
grep "ERROR" app.log | wc -l
grep "ERROR" app.log | sort | uniq -c | sort -nr
```

Name **one command** you would use in a pipeline.

Activity 4: design a pipeline

Goal

Create a file called `error-summary.txt` that contains each distinct `ERROR` line and its count.

Available tools

`grep`, `sort`, `uniq -c`, `wc -l`, `>`, `|`

Write your command, then compare with another pair.

Concept 11: Exit status tells scripts what happened

Key idea

After a command runs, it returns a small number called an exit status. In Unix-style shells, `0` usually means success and nonzero usually means failure.

- `command not found`: the shell could not locate the command.
- `permission denied`: the file exists, but access is not allowed.
- `no such file or directory`: the path does not point to an existing item.

Demo 9: inspect success and failure

Important detail

The output and the exit status are related, but they are not the same thing.

```
ls app.log
echo $?
ls missing.log
echo $?
grep "ERROR" missing.log | wc -l
echo $?
set -o pipefail
grep "ERROR" missing.log | wc -l
echo $?
```

Question

What changes after `set -o pipefail`?

Without `set -o pipefail`, `grep ERROR missing.log | wc -l` exits with status:

A 0

B 1

C 2

D It varies by shell

Concept 12: Error messages are clues, not noise

Key idea

Every error message is a clue. Read it before doing anything else.

1. **Read the error message** — the shell tells you exactly what failed and often why.
2. **Check the manual** — `man <command>` or `<command> --help` explains every option.
3. **Search online** — paste the exact error text; include the command name.
4. **Ask AI** — give context: what you ran, what you expected, what you got.

Order matters

Most errors are self-explanatory once you slow down and read them.

Demo 10: from error to answer

Live on Terminal

Reproduce an error, then use two strategies to investigate it.

```
ls missing.txt
man ls
ls --help
```

Search and AI

- **Online**: search the exact error text plus the command name.
- **AI**: "I ran `ls missing.txt`. I expected a file listing. I got: `ls: cannot access 'missing.txt': No such file or directory`. What could cause this?"

What should feel different after today?

- You can ask: **where am I?**
- You can inspect: **what files exist and what permissions do they have?**
- You can recognise: **which process or command produced this result?**
- You can explain: **why did this command run or fail?**
- You can combine tools: **how can I answer this question from text?**
- You can unblock yourself: **what to do when a command fails.**

The bigger idea

The shell turns computing actions into visible, repeatable, explainable steps.

Exit ticket

1. What does `pwd` tell you?
2. What is one difference between `cp` and `mv`?
3. Why might `./scripts/run.sh` work when `run.sh` does not?
4. What does a pipe connect?
5. What does exit status `0` usually mean?
6. Name two ways to look up what an unfamiliar flag does.

Reflection: Which shell idea do you want to practise more this week?

Poll Everywhere · Word Cloud

pollev.com/fit2109

Type **one command** you will actually use again this week.

Q&A and next steps

Questions?

Before next time

Practise the Week 1 applied tasks until you can explain both the command and the output.